© 2025 Kaiyao Ke

A NOVEL APPROACH TO REPAIR NON-IDEMPOTENT-OUTCOME TESTS WITH AN LLM-BASED AGENT

BY

KAIYAO KE

THESIS

Submitted in partial fulfillment of the requirements
for the degree of Master of Science in Computer Science
in the Graduate College of the
University of Illinois Urbana-Champaign, 2025

Urbana, Illinois

Adviser:

Professor Darko Marinov

ABSTRACT

Flaky tests, characterized by inconsistent results across repeated executions, present significant challenges in software testing, especially during regression testing. Recently, there has been emerging research interest in non-idempotent-outcome (NIO) flaky tests—tests that pass on the initial run but fail on subsequent executions within the same environment. Despite progress in utilizing Large Language Models (LLMs) to address flaky tests, existing methods have not tackled NIO flaky tests. The limited context window of LLMs restricts their ability to incorporate relevant source code beyond the test method itself, often overlooking crucial information needed to address state pollution, which is the root cause of NIO flakiness.

This thesis introduces NIODebugger, the first framework to utilize an LLM-based agent to repair flaky tests. NIODebugger features a three-phase design: detection, exploration, and fixing. In the detection phase, dynamic analysis collects stack traces and custom test execution logs from multiple test runs, which helps in understanding accumulative state pollution (for example, a counter is 2 during the first rerun and becomes 3 in the second rerun). During the exploration phase, the LLM-based agent provides instructions for extracting relevant source code associated with test flakiness. In the fixing phase, NIODebugger repairs the tests using the information gathered from the previous phases. NIODebugger can be integrated with multiple LLMs, achieving patching success rates ranging from 11.63% to 58.72%. Its best-performing variant, NIODebugger-GPT-4, successfully generated correct patches for 101 out of 172 previously unknown NIO tests across 20 large-scale open-source projects. We submitted pull requests for all generated patches; 58 have been merged, only 1 was rejected, and the remaining 42 are pending. The Java implementation of NIODebugger is provided as a Maven plugin accessible at `https://github.com/kaiyaok2/NIOInspector`.

This thesis is based on the paper "NIODebugger: A Novel Approach to Repair Non-Idempotent-Outcome Tests with LLM-Based Agent" accepted to ICSE '25.

To my wife, my parents, my friends, and all the wonderful faculty at Illinois.

ACKNOWLEDGMENTS

I would like to express my deepest gratitude to my advisor, Professor Darko Marinov, for his extraordinary support. When I entered the program, I had little idea what real software engineering research was. Darko patiently taught me every fundamental element of the field. During my first semester, I made virtually no research progress—still mispronouncing "ICSE" by the end of it—yet he never lost faith in me and continuously encouraged me to keep going.

Darko makes even the most intense workdays enjoyable, often breaking the tension with his legendary cracked jokes—many of which still make me laugh when I share them with friends. Far from the stereotypical senior professor high in the clouds, he would stay up late to show me how to work more efficiently, starting from the very basics, like writing shell scripts. I'll never forget how he spent multiple nights during Thanksgiving week helping me revise a paper he wasn't even co-authoring. His integrity—embodied by his motto, "research is 100% or 0%"—is something I deeply admire. His unmatched expertise and his willingness to support me with everything from coursework to presentation skills have been one of the greatest blessings of my academic journey.

I would also like to thank Professor Reyhaneh Jabbarvand for her invaluable mentorship throughout several research projects. Her hands-on guidance and insightful feedback helped shape me into a more capable and independent researcher. I'm also deeply grateful to Professor Lingming Zhang, whose support transformed a simple course project into one of my most meaningful research contributions thus far. His class remains one of my all-time favorites.

To the senior students I've worked closely with—Sam (Runxiang) Cheng and Ali Reza Ibrahimzada—thank you. Collaborating with you under tight deadlines was both intense and incredibly rewarding. I'm also thankful to Yang Chen for the insightful feedback on the original ICSE '25 paper.

A heartfelt thank you to Professor Mariana Silva, who taught the Numerical Methods course I supported as a staff member for four semesters. She is the most enthusiastic and approachable professor I've ever met. Her joyful "happy hours" created lasting memories and sparked lifelong friendships - I'm especially grateful to Yuxuan Chen, Kangyu Feng, and Victor Zhao for welcoming me to the CS 357 family and persistently convincing me how amazing Mariana is. Thanks also to Kriti Chandak, Apramey Hosahalli, Jilian Nylund, Kajal Patel, and more —holding joint in-person office hours and chatting about life, sports,

and travel when no students were around created some of the most fun and memorable moments of my time here.

To my parents—thank you for your unwavering support and encouragement. Your belief in me has been the foundation of my confidence. I am also deeply grateful to my grandparents, whose constant encouragement and genuine curiosity about my research have always brought me extra joy. Their happiness in seeing me pursue my dreams means a lot to me.

To Ryu Okubo and Matt Ma—thank you for being my closest friends in this small town. Escaping my laptop to play Mahjong with you two created some of the happiest memories of grad school.

Most importantly, I would like to express my deepest appreciation to my beautiful wife, Feiqian Yang. Adjusting to life as a graduate student wasn't always smooth, but she has been my rock—offering endless love and support through every high and low. When I thought about giving up on research, she convinced me to push through. She flew across the globe just to walk the campus with me, and stayed up until 7 a.m. helping polish figures for paper drafts I should've finished days earlier. Without her, none of this would have been possible. Losing our final 2-vs-2 billiard games at the frat house was tough—but when you visit, we'll still be one of the top pool duos at Illinois! I am forever grateful for the life and memories we (and our cats) continue to build together.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1
1.1 Problem 1
1.2 Overview of Contributions 2

CHAPTER 2 RELATED WORK 4
2.1 NIO Flaky Tests 4
2.2 LLM-Based Flaky Test Fixing Techniques 4
2.3 LLM-Based Agents 5

CHAPTER 3 APPROACH 6
3.1 Overview 6
3.2 Detection Phase 6
3.3 Exploration Phase 10
3.4 Fixing Phase 15
3.5 Optional Reflection Phase 17

CHAPTER 4 EVALUATION 19
4.1 Research Questions Overview 19
4.2 Evaluation Setup 19
4.3 RQ1: Effectiveness, Generalizability, and Baseline Comparisons in Fixing NIO Tests 20
4.4 RQ2: Contributions to Real-World Software 22
4.5 RQ3: Contributions of Key Components 23

CHAPTER 5 THREATS TO VALIDITY 27

CHAPTER 6 LIMITATIONS 28

CHAPTER 7 CONCLUSION 31

REFERENCES 32

CHAPTER 1: INTRODUCTION

1.1 PROBLEM

Flaky tests, as documented in numerous studies [1, 2, 3, 4, 5, 6, 7, 8], exhibit inconsistent outcomes upon repeated executions of the same version of the code. This inconsistency stems from various factors, such as pollution of shared states. Often, flaky tests originate from pre-existing issues within the codebase which predates recent code modifications. Consequently, their unpredictable behavior poses significant challenges in regression testing. Failures of previously passed tests after code changes may potentially lead developers to draw erroneous conclusions regarding the introduction of new bugs [9, 10]. Furthermore, flaky tests can obscure genuine defects since they typically fail under specific, often uncommon, circumstances [11]. To address this issue, numerous research endeavors have concentrated on automating the prediction [12, 13, 14, 15, 16, 17], detection [18, 19, 20, 21], and mitigation [22, 23, 24, 25, 26] of specific types of flaky tests.

Order-dependent (OD) flaky tests [27] represent one extensively studied category. OD tests exhibit deterministic passing behavior in some test orders but fail in others. This behavior arises from undesirable dependencies between tests, often unnoticed due to implicit ordering among prevailing implementations of testing frameworks, like JUnit [28] for Java. Shared state pollution is the dominant source of OD flakiness, where "polluter" tests modify states shared among tests, causing "victim" tests to fail when running after them [18]. Detecting OD tests is crucial, particularly with respect to the resilience of test suites to framework updates or migration to advanced regression testing techniques such as test prioritization, selection, and parallelization.

Exploring all possible orders of test execution can be excessively time-consuming, especially for large test suites. Therefore, one strategy for detecting OD tests entails identifying latent-polluters and latent-victims within test suites. Latent-polluters are any tests that modify shared states without restoring them, while latent-victims are any tests that rely on deterministic shared states. A recent study [29] has revealed that only a small portion of latent polluters and victims are convertible into actual test order dependency, as they mainly involve non-public states or have negligible impact. In response, the study proposed focusing on non-idempotent-outcome (NIO) tests [29], which exhibit changes in test outcomes across repeated runs due to their pollution of certain shared states. An NIO test is both a latent-victim and a latent-polluter and can be easily identified by running tests twice in the same environment. Detecting NIO tests can aid in preemptively addressing OD test

Figure 1.1: A Sample NIO Test

```
1 void t1() { assertEquals(0, w); w = 1; } // NIO Test
2 void t2() { assertEquals(0, w); ... }
3 void t3() { w = 1; ... }
```

issues, because an NIO test can become OD when a new test pollutes the shared state, and can turn a new test into OD if it reads a part of the NIO-polluted state. For example, in Figure 1.1, `t1()` is an NIO test. Consider when `t2()` is added - now `t1()` becomes a real polluter of `t2()`. On the other hand, when `t3()` is added, the NIO test `t1()` becomes a victim of `t3()`. The key to fixing an NIO test is to reset the polluted state before or after test execution, which, in this case, is achieved by resetting `w` to `0`.

In recent years, numerous techniques have emerged to fix program defects and even a subset of flaky tests. Multiple research studies have focused on automatically fixing buggy programs using Large Language Models (LLMs). CodeBERT was the first LLM explored for automatic program repair (APR) [30], while subsequent studies have shown promising results using more advanced LLMs [31]. Among these, GPT models have demonstrated superior performance in APR tasks compared to other LLMs [32]. More recent research on LLM-based bug fixing [33, 34] has emphasized the importance of providing sufficient context, including buggy code, to enhance the performance of LLMs in bug-fixing tasks. This has led to the development of LLM-based agents [35, 36, 37], which treat the LLM as an autonomous agent capable of planning and executing actions to achieve the goal of fixing bugs. As for flaky tests, although non-agentic LLM-based techniques have achieved state-of-the-art results in addressing some types of flaky tests that can be fixed without knowledge of the main code under test [38, 39], they do not generalize well to others [40]. Due to the context window limitations of LLMs, these techniques do not consider rich information from the source code beyond the test code itself. In particular, non-agentic techniques cannot fix NIO tests effectively, because fixing these tests often requires additional knowledge to properly clean up state pollution.

1.2 OVERVIEW OF CONTRIBUTIONS

This thesis presents NIODebugger, a three-phase approach that uses an LLM-based agent to address NIO tests.

During the detection phase, NIODebugger reruns tests in an isolated environment to record test status and flag potential NIO tests, and also utilizes a custom JUnit summary listener

to detect variations in stacktraces across multiple test runs, which may unveil patterns of state pollution—the cause of NIO tests.

During the exploration phase, NIODebugger retrieves the test code for each NIO test and integrates it with the collected dynamic analysis data. It queries an LLM for instructions to find relevant source code that can assist in fixing the NIO test. NIODebugger equips the agent with numerous code-extracting workflows, allowing it to interact with the codebase for context-specific information, similarly to a human developer.

Following the agent's instructions, the fixing phase of NIODebugger collects the relevant source code and queries an LLM to fix tests. The LLM first generates a patch for the specific test, and then uses the patch and the original test file to generate a compilable file that replaces the original test file.

To evaluate the effectiveness of NIODebugger, we ran the detection phase on a selection of popular open-source projects. After running the detection phase at scale, we identified 172 flaky tests across 20 popular GitHub projects. We utilize four different underlying LLMs (two open-source and two proprietary) for NIODebugger to compare their performance against existing non-LLM-based baselines that can be extended to fix NIO tests. The best variant, NIODebugger-GPT-4, significantly outperforms baseline techniques by successfully fixing 101 of these tests, of which 58 were accepted until now. Only one pull request (PR) was rejected, while the rest are still pending. Patches for 52 tests were directly accepted without modifications, while 6 were accepted after we made the changes requested by the open-source developers, such as moving the cleanup routine to an `@After` method. These results demonstrate the practical applicability and effectiveness of NIODebugger in detecting and repairing flaky tests in real-world open-source projects.

In summary, this thesis contributes the following:

- The first LLM-based agent for flaky test fixes, leveraging the novel approach of using LLMs to provide guidance in searching for relevant context during the fixing process.
- A Java implementation of NIODebugger that effectively detects and fixes NIO flaky tests. The tool is published on Maven Central.
- A framework that incorporates dynamic analysis during the detection phase, enabling the detection and fixing of flaky tests to occur within a single lifecycle.
- A dataset comprising previously unidentified NIO flaky tests found in popular open-source projects, along with their corresponding auto-generated patches produced by NIODebugger, if applicable.

CHAPTER 2: RELATED WORK

2.1 NIO FLAKY TESTS

The sole study on non-idempotent-outcome (NIO) flaky tests [29] conducted a comprehensive analysis across open-source projects, identifying a total of 223 NIO Java tests. Utilizing the iDFlakies tool [18] tailored for detecting order-dependent (OD) flaky tests, the study facilitated the detection of Java NIO tests by test repetition within a single execution. The study also manually fixed the majority of the identified tests. Despite its significant contribution, the study faced two notable limitations:

- Unspecialized Detector with Limited Accessibility: Its detector, derived from modifications to the existing iDFlakies tool, is restricted to supporting JUnit 4 and lacks the code conciseness and optimal efficiency needed for NIO test detection. Besides reporting possible NIO tests, it does not produce further information for debugging.
- Manual Fixes: All test fixes were performed manually, demanding in-depth understanding of the source code and consuming significant time resources. Previous study shows that a fix to one NIO test usually requires at least an hour from multiple authors [29].

These limitations call for a more specialized and accessible solution to the mitigation process for NIO tests.

2.2 LLM-BASED FLAKY TEST FIXING TECHNIQUES

Though there is no previous work that fixes NIO flaky tests automatically, LLM-based techniques have achieved state-of-the-art results in addressing other categories of flaky tests. FlakyDoctor [38] presents an approach that directly queries LLMs to fix flaky tests by providing error messages from failed test runs. FlakyDoctor is robust at fixing order-dependent (OD) and implementation-dependent (ID) tests (tests making false assumptions on underdetermined APIs) [41]. It extracts these error messages from executions of the ID test detection tool NonDex [19] or the OD test detection tool iDFlakies [18]. Although the FlakyDoctor framework could potentially be extended to resolve NIO tests, its current approach is limited for 3 major reasons. Firstly, FlakyDoctor only includes the code of the flaky test method when prompting the LLM, and this does not generalize well to fixing NIO tests, which often

require invoking custom cleaner methods defined in main classes. Secondly, it treats the failure message of a test as constant. While this approach aligns with most other types of flaky tests, it may not accurately capture the behavior of NIO tests. For instance, consider an NIO test that increments a static variable initialized at `0` and then asserts it to be `1`. In successive test runs, this test may yield different error messages (e.g., "`expected:<1> but was:<2>`" and "`expected:<1> but was:<3>`"). The variability in error messages across successive runs provides valuable insights for LLMs to understand errors stemming from the accumulation of state pollution, including incremented counters, retrieval of different objects from the head of a collection, and other factors. Furthermore, given the extensive research on ID and OD flaky tests, FlakyDoctor's zero-shot architecture has shown promising results. However, the landscape differs for NIO tests, with only one previous study conducted in 2022.

FlakyFix [39] introduces an innovative framework that leverages neural networks to predict fix categories before utilizing LLMs to address flaky tests. The authors use publicly reported flaky tests from the IDoFT [42] dataset to generate labeled datasets for 13 heuristically defined fix categories. They train a model to predict the fix category using the flaky test code before directly employing LLMs to fix the test. While FlakyFix could also potentially be extended to include NIO-related fix categories, its effectiveness is also limited by its reliance solely on test code for prompting and its omission of valuable information from stack traces.

2.3 LLM-BASED AGENTS

LLM-based agents represent a new line of research in automated program repair (APR). This approach augments the LLM into an agent capable of autonomously planning and executing actions to fix bugs by invoking suitable tools or APIs. It leverages the LLM's ability to understand the root cause of issues and effectively retrieve context. RepairAgent [35] is the first work to address the program repair challenge using an LLM-agent-based technique, applied to the Defects4J [43] dataset. AutoCodeRover [37] introduces an LLM-agent-based approach for resolving pending GitHub issues to autonomously achieve program improvement. FixAgent [36] proposes the first automated, unified debugging framework via LLM agent synergy, where two LLM agents act as a bug localizer and program repairer. These agents are prompted to explicitly track key variables at critical points in the buggy program and discuss how such tracking guides their task completion. Additionally, they help construction of the program context concerning its specifications and dependencies.

There is no agent-based approach in the field of flaky test repair. This thesis is the first to incorporate such a design to glean source code that can inspire patching NIO tests.

CHAPTER 3: APPROACH

3.1 OVERVIEW

Figure 3.1 provides an overview of NIODebugger, a three-phase framework designed to tackle non-idempotent-outcome (NIO) flaky tests by an LLM-based agent. In the detection phase, NIODebugger reruns tests within the same environment to pinpoint potential NIO tests while simultaneously performing dynamic analysis to capture essential information useful for debugging. In the exploration phase, NIODebugger combines the identified test code with dynamic analysis data and consults an LLM for guidance on locating relevant source code. This equips the LLM with code-extracting techniques, enabling it to retrieve context necessary for rectifying NIO tests. In the fixing phase, NIODebugger gathers the pertinent source code as directed by the previous phase and utilizes an LLM to generate patches. Finally, an optional reflection phase is available to iteratively refine the patch based on previous results. This approach effectively addresses NIO test flakiness by leveraging dynamic analysis insights, the gathered information, and a one-shot learning example.

The majority of flaky tests identified in the previous study [29] originate from open-source Maven Java projects, reflecting Maven's status as the preferred build tool for large-scale software applications due to its robust dependency management and build handling capabilities. A recent study [44] highlights that over 76% of Java developers use Maven for their builds. Similarly, JUnit is widely adopted, with approximately 85% of Java developers using it as their unit testing framework, according to a recent survey [45]. In light of these trends, we developed NIODebugger as a command-line plugin available on Maven Central, specifically designed to detect and address JUnit NIO tests in Maven Java projects. However, the NIODebugger technique is not limited to Java, Maven, or JUnit. The three-phase workflow and LLM agent can be generalized to support other programming languages, build tools, and testing frameworks. By customizing the detection phase to be framework-specific and tailoring the exploration and fixing phases to be language-specific, NIODebugger can be adapted to a wide range of environments.

3.2 DETECTION PHASE

To address the key limitations of the detector used in the original study of NIO flaky tests [29], which lacked useful debugging information, we developed a specialized workflow for detecting NIO tests. Algorithm 3.1 illustrates our systematic approach involving dynamic

Algorithm 3.1: NIO Flaky Tests Detection and Dynamic Analysis

Inputs: Project P, Number of Reruns *numReruns*
Output: Possible NIO Flaky Tests *possibleNIOTests*, Other Flaky Tests *otherFlakyTests*, Dynamic Analysis Output *dynamicAnalysisLog*

```
1  firstRunResult, rerunResults ← {};
2  rerunStacktraces, rerunExtraInfo ← {};
3  possibleNIOTests, otherFlakyTests ← ∅;
4  allTests ← findAllTests(P);
5  foreach t ∈ allTests do
6      runner ← createIsolatedTestRunner(t);
7      env ← createIsolatedEnvironment(t);
8      result ← runTest(runner, env, t);
9      firstRunResult[t] ← result;
10     rerunResults[t] ← [];
11     rerunStacktraces[t] ← [];
12     rerunExtraInfo[t] ← [];
13     for i ← 1 to numReruns do
14         result, extraInfo ← runTestWithExtraInfoCollector(runner, env, t);
15         rerunResults[t].append(result);
16         rerunExtraInfo[t].append(extraInfo);
17         if result = fail then
18             stacktrace ← getStackTrace(env);
19             rerunStacktraces[t].append(stacktrace);
20 foreach t ∈ allTests do
21     if (firstRunResult[t] = pass) ∧ (∀result ∈ rerunResults[t], result = fail) then
22         possibleNIOTests ← possibleNIOTests ∪ {t};
23     else if ∃result1, result2 ∈ (firstRunResult[t] ∪ rerunResults[t]), result1 ≠ result2 then
24         otherFlakyTests ← otherFlakyTests ∪ {t};
25 foreach t ∈ possibleNIOTests do
26     runner ← getIsolatedTestRunner(t);
27     env ← createCleanIsolatedEnvironment(t);
28     result ← runTest(runner, env, t);
29     if result = fail then
30         possibleNIOTests ← possibleNIOTests \ {t};
31         otherFlakyTests ← otherFlakyTests ∪ {t};
32         continue;
33     for i ← 1 to numReruns do
34         result ← runTest(runner, env, t);
35         if result = pass then
36             possibleNIOTests ← possibleNIOTests \ {t};
37             otherFlakyTests ← otherFlakyTests ∪ {t};
38             break;
39 dynamicAnalysisLog ← log(possibleNIOTests, rerunStacktraces, rerunExtraInfo);
40 return possibleNIOTests, otherFlakyTests, dynamicAnalysisLog;
```

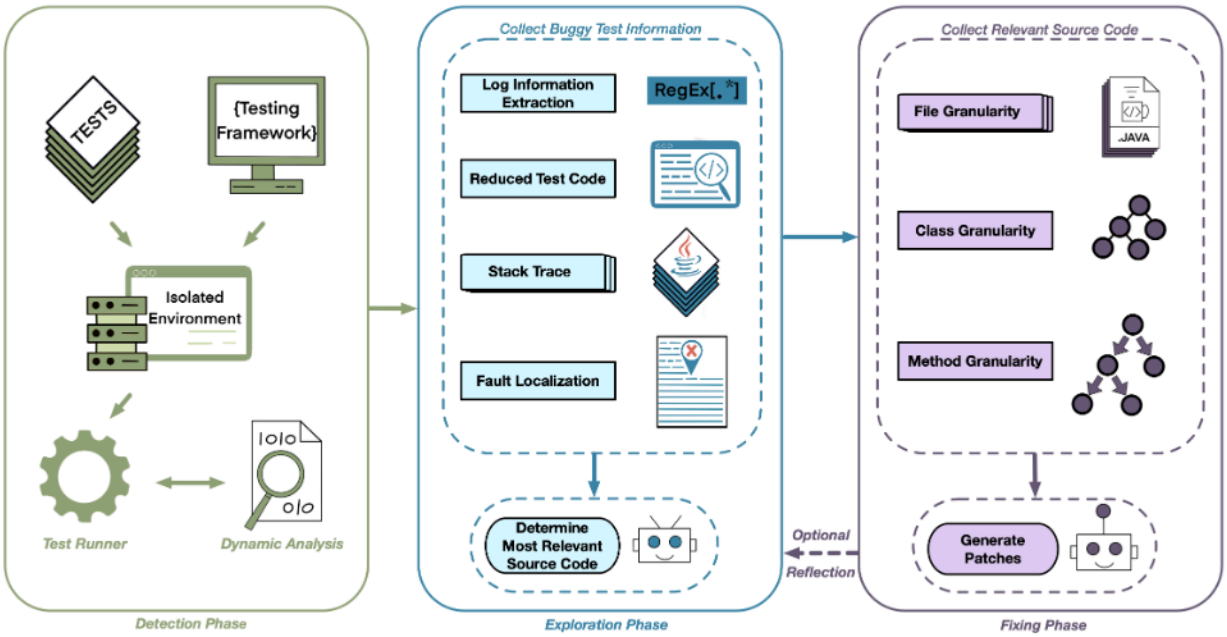


Figure 3.1: NIODebugger Architecture

analysis. Initially, NIODebugger identifies all tests in a given project. For each test, it first creates an isolated testing environment and runs the test once, recording this initial result. It then enters a loop to rerun the test multiple times within the same environment. During each rerun, NIODebugger captures the result of the test execution. If a test fails, it retrieves and stores the corresponding stack trace. Regardless of the test's result, additional runtime information, such as logging outputs and warnings, is also collected. After all reruns are completed for each test, NIODebugger assesses whether the test qualifies as a possible NIO flaky test. This determination is based on the observation that if a test passed on the first run but consistently failed in subsequent reruns, it is flagged as a potential NIO flaky test. For each test flagged as a potential NIO flaky test, NIODebugger re-creates a clean isolated environment and reruns the test to confirm whether it consistently exhibits non-idempotent behavior. The detection phase concludes by logging the identified NIO tests along with the results of the dynamic analysis.

In addition, we describe our efforts to integrate the algorithm seamlessly with the modern Java ecosystem. In Maven projects, the Maven Surefire Plugin [46] is commonly used during the build lifecycle to execute JUnit tests. However, Surefire automatically spawns and terminates forked Java Virtual Machines (JVMs) for testing, making it challenging to rerun tests in the same JVM. To address this limitation for Maven Java projects, the Java implementation of the detection phase initializes a custom isolated class loader to load all

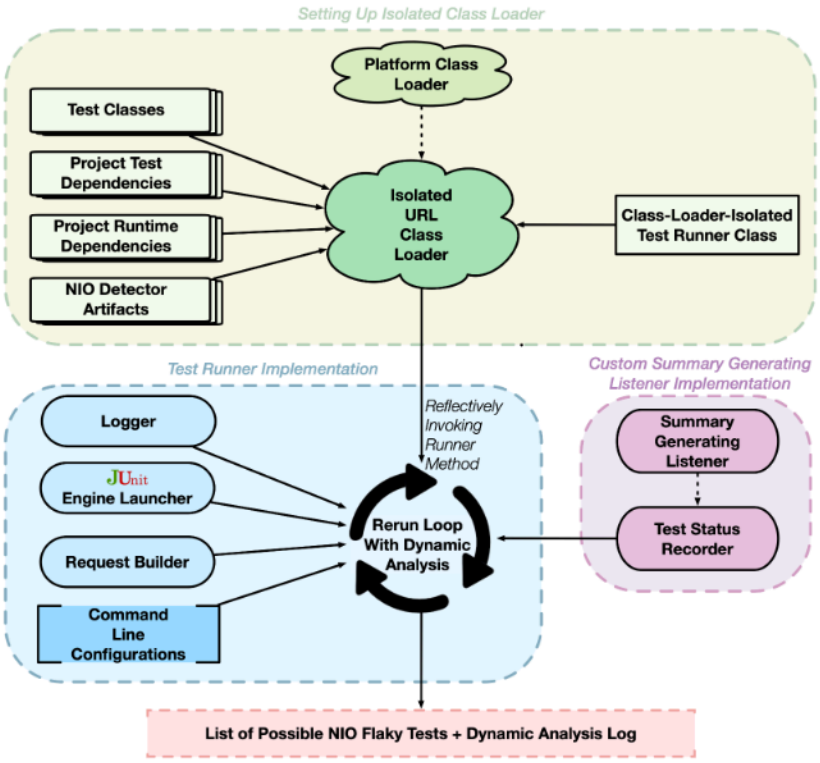


Figure 3.2: Java Implementation of Detection Phase

test classes, necessary artifacts for test execution, and the JUnit test engine. Subsequently, it employs a class-loader-isolated JUnit runner capable of rerunning tests within the same JVM. While the workflow is tailored to Java, its underlying principles can be adapted to other programming languages.

Figure 3.2 shows the workflow of the detection phase for JUnit Java tests. Specifically, the core detection architecture includes four major components:

1. **Isolated Class Loader**: To ensure a highly streamlined environment that includes only the essential dependencies necessary for repeated unit test execution, the isolated class loader minimizes the number of dependencies. The class loader provides a concise environment encompassing test classes, essential runtime dependencies, specialized unit-testing artifacts, NIODebugger's proprietary classes, and the entire Java standard library. Notably, the bootstrap class loader in Java versions 9 and later exclusively loads core Java classes from the runtime environment, such as those within the `java.lang` package. However, it does not load packages like `java.sql` which require the `--add-module` flag during JVM launch. To ensure comprehensive access to Java SE platform APIs, including JDK-specific runtime classes, we configure the parent of

our isolated class loader to be the platform class loader. We implement the isolated class loader as a `URLClassLoader`, facilitating the loading of artifacts via path URLs.

2. **Class-Loader-Isolated Test Runner**: With the class loader prepared, the runner can rerun specified test classes or methods, or the entire test suite by default. Using reflection, the runner invokes the JUnit launcher factory's creator method from within the isolated class loader. The request builder, which accepts designated test classes or methods identified by JUnit discovery selectors, is passed to the test engine launcher. The launcher activates the appropriate JUnit Engine, which processes the request and orchestrates the repetitive execution of tests. The runner employs a custom logger to capture useful test information for debugging. During this process, the test runner reruns unit tests, examines their outcomes, and flags tests that pass initially but fail in subsequent runs.

3. **Custom Summary Generating Listener**: NIODebugger uses an extension of JUnit's summary generating listener to provide a comprehensive overview of test execution. While the JUnit listener only captures the names of failed tests, our custom listener maintains a map of test statuses, updated with each test's completion. This map ensures that the runner has information about tests that pass initially but fail later, and also supports the extraction of additional dynamic information such as stack traces and warnings.

4. **Executor**: An executor operates on top of the three components described above. Implemented as a Maven Mojo, it is responsible for executing tasks within the Maven build process. Our rerun execution Mojo identifies all tests slated for execution, processes command-line inputs, retrieves URLs for the isolated class loader, and loads the class-loader-isolated test runner class into the customized class loader. Finally, it invokes the JUnit runner method via reflection.

3.3 EXPLORATION PHASE

The exploration phase of NIODebugger is a two-step process designed to utilize our LLM-based agent for NIO flaky tests. Initially, it performs static analysis on the results from the detection phase to gather the most relevant information for each test. Subsequently, the relevant information, along with the test code, is used to query an LLM for suggestions on relevant code that may aid in debugging. This phase can be easily generalizable to any programming languages and testing frameworks. Part of Figure 3.3 illustrates the

Algorithm 3.2: Exploration Phase of NIODebugger

Inputs: Project P, Possible NIO Flaky Tests *possibleNIOTests*, Dynamic Analysis Log *dynamicAnalysisLog*, Number of Reruns *numReruns*
Output: Instructions for Relevant Source Code Extraction *instructions*

```
1  instructions, errorLineNums, errorLines ← {};
2  stackTraces, extraInfo, reducedTestCode ← {};
3  foreach t ∈ possibleNIOTests do
4      testFileCopy ← collectTestFile(t, getCodeBase(P));
5      errorLineNums[t] ← [];
6      errorLines[t] ← [];
7      for i ← 1 to numReruns do
8          errorLineNum ← getStackTrace(dynamicAnalysisLog, t);
9          errorLineNum ← errorLineNum.getErrorLineNum(i);
10         errorLineNums[t].append(errorLineNum);
11     foreach lineNum ∈ errorLineNums[t] do
12         errorLine ← extractLine(testFileCopy, lineNum);
13         errorLines[t].append(errorLine);
14     foreach method ∈ getAllTestMethodsFromFile(testFileCopy) do
15         if method.getName() ≠ t.getName() then
16             testFileCopy ← removeMethod(testFileCopy, method);
17     reducedTestCode[t] ← testFileCopy;
18     extraInfo[t] ← parseExtraInfo(dynamicAnalysisLog, t);
19     stackTraces[t] ← [];
20     for i ← 1 to numReruns do
21         stackTrace ← getStackTrace(dynamicAnalysisLog, t);
22         stackTrace ← stackTrace.extractStackTraceAtRerunNum(i);
23         stackTraces[t].append(stackTrace);
24 generalDescription ← getGeneralDescriptionOfNIOTests();
25 foreach t ∈ possibleNIOTests do
26     prompt ← buildResponseFormatConstrainedPrompt(
         t, errorLines[t], stackTraces[t], reducedTestCode[t],
27         extraInfo, generalDescription);
28     instruction ← queryLLM(prompt);
29     instructions[t] ← instruction;
30 return instructions;
```

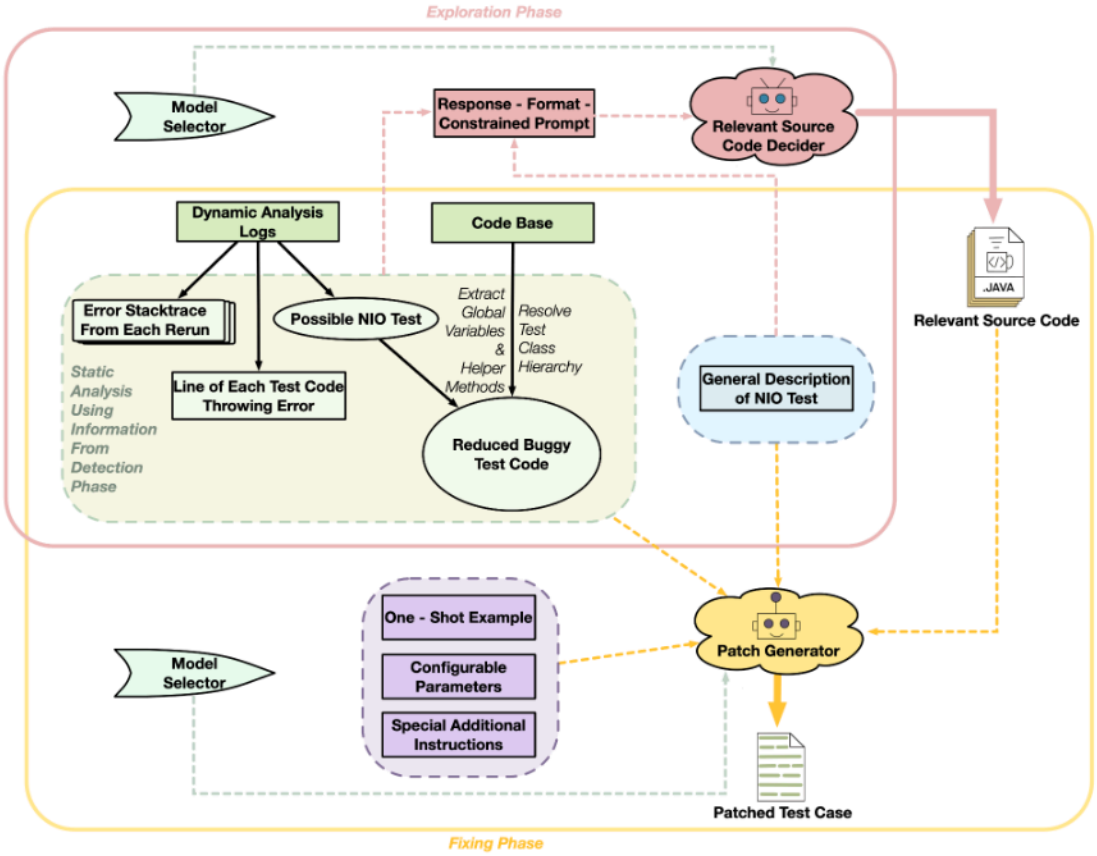


Figure 3.3: Exploration and Fixing Phases of NIODebugger

overall workflow of the exploration phase, while Algorithm 3.2 details the specific process of interacting with an LLM-based agent to generate instructions for relevant source code extraction.

Specifically, for each test identified as a potential NIO flaky test during the detection phase, NIODebugger extracts error line numbers from the dynamic analysis log and parses the corresponding test file to locate these lines. It then creates a reduced version of the test file, adhering to the principle that unit tests should be independent. To minimize noise and manage context length, NIODebugger removes other test methods while retaining helper methods, fields, and classes within the file. Additionally, it includes inherited fields and methods from superclasses, resulting in a streamlined yet informative codebase for each NIO test. Meanwhile, NIODebugger parses extra information from the dynamic analysis log, including stack traces from various reruns. All collected data, along with a general description of NIO tests, is fed into an LLM agent, which then decides to extract relevant code from the repository that assists in fixing the NIO test.

To query the LLM agent, we construct a **response-format-constrained prompt**, which includes explicit instructions on the desired structure and content of the LLM's response. The prompt includes guidelines or templates which the LLM must adhere to in generating

its response, thereby ensuring that the output is consistent with the specified format to invoke a **parameterized workflow**. The response-format-constrained prompt not only contains test-specific information, but also includes a section that specifies all custom source-code-searching parameterized workflows from which the LLMs can choose, detailing the inputs to be passed into these workflows and the format of the LLM response to ensure it is automatically parsable for invoking such workflows. Figure 3.4 shows the format of the response-format-constrained prompt. Below is a detailed overview of all parameterized workflows:

1. **Find Code of a Specific Method**: This workflow is used when the LLM identifies that a method might be, or is associated with, a latent polluter or cleaner, and requires access to the method's implementation for verification or additional relevant information. NIODebugger requires the LLM to respond `Find Method Code: {className.methodName}` in this scenario.

2. **Find Code of a Specific Class**: This workflow is employed when the LLM determines that understanding the implementation of a type as a whole is essential, especially when it needs a comprehensive view of a class's potential fields that may be shared and polluted. NIODebugger requires the LLM to respond `Find Class Code: {className}` in this scenario.

3. **Function Name Inference and Code Matching**: Used when the LLM anticipates potential functions to clean up states but does not know in advance which class defines such methods. The LLM can then infer a function name based on the desired functionality of state cleaning and locate code for multiple functions with names similar to the guessed name. This feature can also be utilized when the LLM needs to explore a function that appears in the code but does not know which class defines the function, such as in long call chains. NIODebugger requires the LLM to respond `Find Hypothesized Method: {possibleMethodName}` in this scenario.

4. **Explore File With Similar Names**: This workflow is utilized when the LLM is unable to make a specific decision at the class or method level and opts to explore source files whose names are similar to the file containing the NIO test, potentially uncovering more insights. NIODebugger requires the LLM to respond `Find Relevant File` in this scenario.

5. **No Source Code Needed Beyond Reduced Test Code**: If the reduced test code contains enough information for a fix, NIODebugger can skip further exploration of

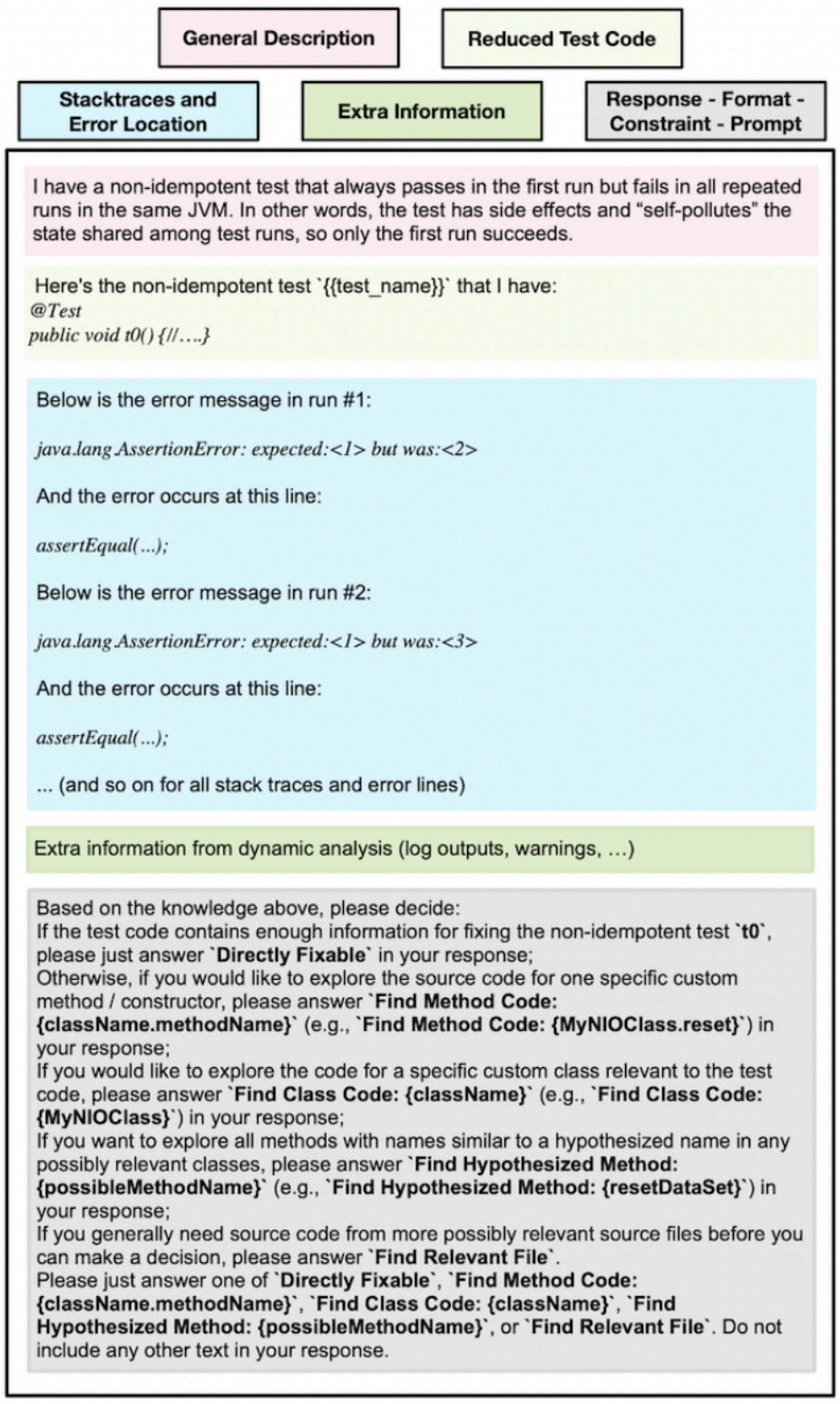


Figure 3.4: Response-Format-Constrained Prompt for Relevant Source Code Exploration

the code base. NIODebugger requires the LLM to respond `Directly Fixable` in this scenario.

3.4 FIXING PHASE

The fixing phase of NIODebugger is follows instructions from exploration phase to extract relevant source code information and query an LLM for generating a final patch for each test. Part of Figure 3.3 illustrates the architecture of the fixing phase, while Algorithm 3.3 outlines the workflow of this phase. It takes as input the maximum number of characters allowed in the collected source code (to fit within the context window), project source code, a set of all possible NIO tests, and a dictionary of instructions from the exploration phase. In this phase, NIODebugger iterates over all project source files, extracting their code and constructing an Abstract Syntax Tree (AST) [47] for each file. By performing a Depth-First Search (DFS) on the AST, NIODebugger identifies and stores the code for each method and class. This step is specific to the language used; for Java, we utilized JavaParser [48], but similar functionality is available in other languages, such as the `ast` module [49] for Python and Clang [50] for C++.

For each NIO test, NIODebugger parses the instruction from the exploration phase to determine the required workflow. NIODebugger retrieves the relevant source code from the constructed maps when an exact match is found, or identifies the most similar entries using the Levenshtein distance [51], a common metric to map LLM-generated outputs to executable tools in agent-based program repair [35]. The relevant source code is then written unless the agent decides that no additional source code is needed. Following the source code extraction, the fixing phase synthesizes a comprehensive prompt for each possible NIO test, including a general description of NIO tests, the test method name, the reduced source code with the test method, stack traces from multiple test reruns, instructions for fixing NIO flaky tests, a one-shot example, and additional information from previous phases. Additionally, the fixer supports custom requirements for fix generation, such as specifying "fix the method itself without adding `setUp()` or `tearDown()` methods." Figure 3.5 illustrates the final prompt to query a patch. As the input test file is streamlined to exclude other test methods, the patch contains only the relevant test method and its associated helper methods or classes.

During post-processing, we provide the LLM with both the original test file and the patch for one method, and prompt it to generate a compilable version to replace the original file. The separation between the fixer phase and code replacement is based on the intuition that the fixer should not be exposed to extraneous code (e.g., other test methods) when formulating a patch for one test. Additionally, this approach allows optional manual verification

Algorithm 3.3: Fixer Phase of NIODebugger

Inputs: Maximum number of characters allowed in collected source code n, Project P, Set of all Possible NIO Tests *possibleNIOTests*, Dictionary of instructions *instructions*

Output: Set of patches *patches*

```
fileCodeMap, classCodeMap, methodCodeMap, patches ← {};
generalDescription ← getGeneralDescriptionOfNIOTests();
errorLines, stackTraces, extraInfo, reducedTestCode ← getInfoFromPreviousPhases();
oneShotExample ← getOneShotExample();
foreach file ∈ findAllSourceFiles(P) do
    fileCodeMap[file] ← extractCode(file);
    AST ← getAST(file);
    foreach node ∈ performDFS(AST) do
        if isInstance(node, method) then
            methodName ← getFullPathMethodName(node);
            methodCodeMap[methodName] ← extractCode(file, methodName);
        else if isInstance(node, class) then
            className ← getFullPathClassName(node);
            classCodeMap[className] ← extractCode(file, className);
relevantSourceCode ← "";
foreach t ∈ possibleNIOTests do
    workflow, nameToSearch ← parseInstruction(instructions[t]);
    if workflow = "Find Code of a Specific Method" then
        relevantSourceCode ← methodCodeMap[nameToSearch];
    else if workflow = "Find Code of a Specific Class" then
        relevantSourceCode ← classCodeMap[nameToSearch];
    else if workflow = "Function Name Inference and Code Matching" then
        distances ← {};
        foreach key ∈ methodCodeMap do
            distance ← ComputeLevenshteinDistance(nameToSearch, key);
            distances.append((distance, key));
        Sort distances by distance;
        relevantSourceCode ← {};
        foreach _, key ∈ copyFromHeadUntilGettingNCharacters(distances, n) do
            relevantSourceCode.append(methodCodeMap[key]);
    else if workflow = "Explore File With Similar Names" then
        distances ← {};
        foreach key ∈ fileCodeMap do
            distance ← ComputeLevenshteinDistance(nameToSearch, key);
            distances.append((distance, key));
        Sort distances by distance;
        relevantSourceCode ← {};
        foreach _, key ∈ copyFromHeadUntilGettingNCharacters(distances, n) do
            relevantSourceCode.append(fileCodeMap[key]);
    prompt ← buildFixerPrompt(t, errorLines[t], stackTraces[t], extraInfo[t]
       oneShotExample, reducedTestCode[t], generalDescription,
         relevantSourceCode);
    patches[t] ← queryLLM(prompt, n);
return patches;
```

and code patching, potentially reducing the cost of invoking the LLM.

3.5 OPTIONAL REFLECTION PHASE

NIODebugger supports iterative refinement as an optional step, which was employed in our evaluation. If the fixer fails to produce compilable code or to resolve test non-idempotency (i.e., the detection phase still reports non-idempotency after patching), the agent re-performs the exploration and fixing steps. During this phase, the prompt is enhanced with the previous parameterized workflow decisions, the generated patch, and new execution results. While we do not make this step mandatory due to the high computational and monetary costs associated with LLMs, we have configured our Maven plugin to automatically incorporate previous run information into the exploration and fixing phases, whenever available. Additionally, we offer a fully automatic setup that includes up to three reflection runs, which the user can choose to enable.

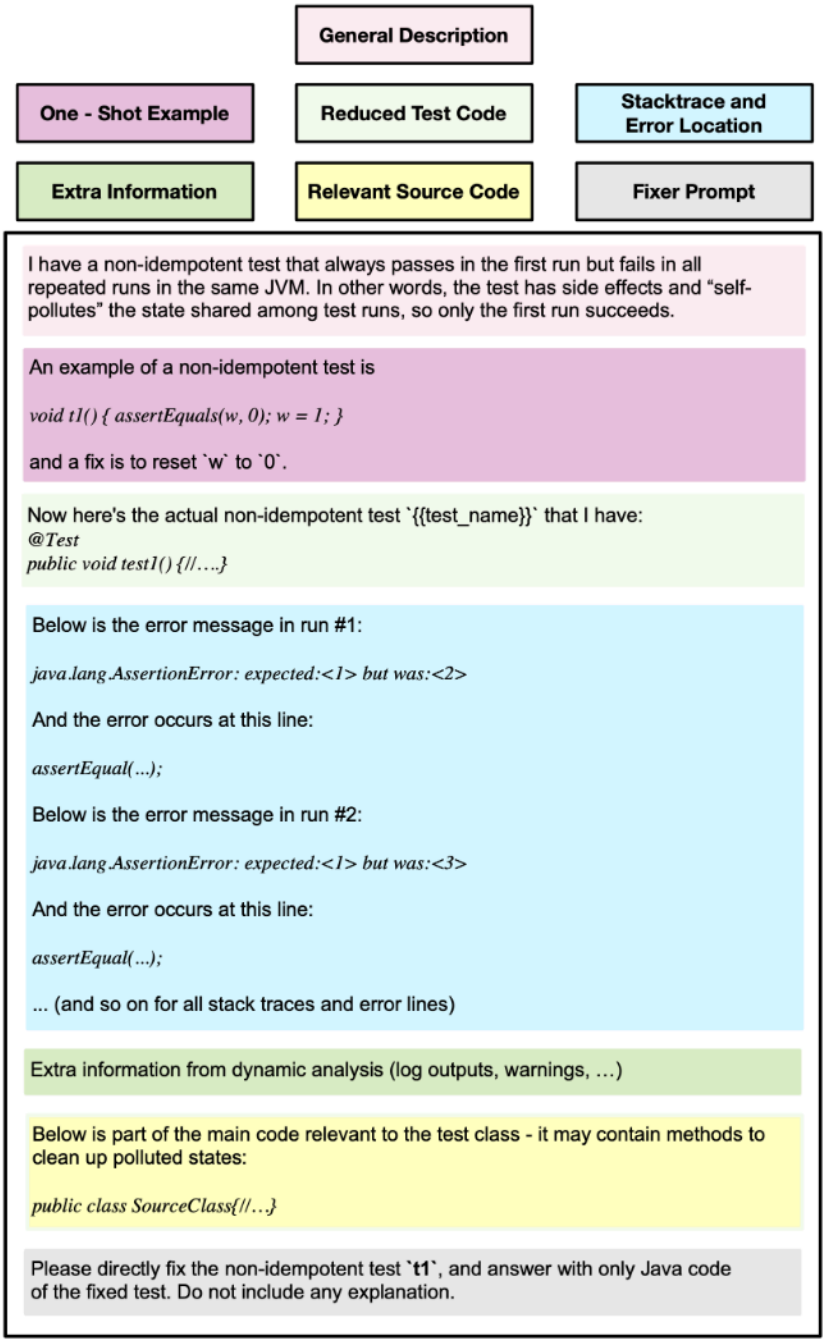


Figure 3.5: Prompt for LLM to Generate a Patch

CHAPTER 4: EVALUATION

4.1 RESEARCH QUESTIONS OVERVIEW

To evaluate the effectiveness of NIODebugger, we investigate the following research questions:

RQ1 Effectiveness, Generalizability, and Baseline Comparisons in Fixing NIO Tests: How effective is NIODebugger in generating patches for NIO tests that preserve the original test logic while eliminating non-idempotency? How generalizable is its effectiveness across different LLMs? Is NIODebugger more effective than existing techniques that could potentially be adapted to address NIO tests?

RQ2 Contributions to Real-World Software: What is the attitude of developers working on large-scale, popular projects toward patches generated by NIODebugger?

RQ3 Contributions of Key Components in NIODebugger: What is the extent of the contribution of the key components in the LLM-based agent?

4.2 EVALUATION SETUP

We used the GitHub API query to obtain a list of popular Java repositories, sorted by the number of stars, with a push within a year. From the query results, we selected the top 300 repositories and filtered them to retain only those containing a `pom.xml` file in the root directory. This process resulted in a final list of 242 repositories, and we successfully executed NIODebugger on 174 of them. The remaining projects failed due to Maven build issues or test hang-ups (unrelated to NIODebugger), or lack of support for Java 9+ (a prerequisite for running NIODebugger).

Due to the time-consuming nature of initializing a separate JVM for each test class or method in large projects, we executed the detection process at the module granularity, using `numRerun = 3`—which spawns one JVM to execute the module's test suite four times consecutively. Note that this method may overlook some NIO tests whose polluted states are resolved by preceding methods. Our script identified 192 potential NIO flaky tests across 21 projects. After that, we reran the detection phase of NIODebugger at the reported test granularity with `numRerun = 10` to verify non-idempotent behavior. We failed to observe non-idempotency in 17 of the 192 tests, and found they were actually order-dependent. For example, if the original order of a test suite is `t1, t2`, then if `t2` modifies a state used by `t1`,

`t1` may pass in the first run but fail afterwards. Another 3 of the reported tests were found to be nondeterministic; they passed in the initial run but failed in subsequent reruns by chance and did not exhibit the same behavior when rerun in isolation. Finally, we confirmed 172 possible NIO tests across 20 projects.

Our goal is to evaluate NIODebugger on all the 172 tests. In line with existing LLM-agent-based program repair techniques [35, 36, 37], we selected GPT-3.5 Turbo and GPT-4 as the two proprietary, API-based state-of-the-art LLMs for the exploration, fixing, and optional reflection phases. To evaluate NIODebugger's generalization capability across multiple LLMs, including open-source models, we also included two top-performing instruction-tuned open-source models, namely DeepSeek-Coder-33B-Instruct [52] and Qwen2.5-Coder-32B-Instruct [53], identified from Aider's Code Editing Leaderboard [54], which ranked models for their ability to generate code edits seamlessly integrated into existing codebase. Note that we selected the instruction-tuned coder variants to address both instruction-following needs during the exploration phase and patch generation in the fixing phase.

Using each LLM, we ran the exploration and fixing phases to generate solutions for each NIO test, also allowing up to 3 reflection runs. We conducted parameter tuning on the temperature setting for each model, utilizing the 149 previously fixed NIO tests recorded by IDoFT [42]. The tuning process began with greedy decoding (temperature = 0) and increased temperature by 0.1 in each iteration, up to a maximum of 2.

Importantly, the 149 tests used for parameter tuning were *entirely disjoint* from the 172 tests used for evaluation, which comprised newly detected, previously unknown tests, ensuring no data contamination. The optimal temperature values found were 0.7 for both GPT models, 0.5 for DeepSeek-Coder-33B-Instruct, and 0.6 for Qwen2.5-Coder-32B-Instruct. For all other parameters, we adhere to the default settings recommended in their documentation as best practice. Additionally, we allowed up to three iterations in the reflection phase, enabling the agent to incorporate insights from the execution results of previous patches. The experiment was conducted using Java 17 on Ubuntu 22.04.3.

4.3 RQ1: EFFECTIVENESS, GENERALIZABILITY, AND BASELINE COMPARISONS IN FIXING NIO TESTS

We ran the aforementioned experiment on the 172 detected NIO tests using each of the four LLMs. A patch is considered correct if it satisfies the following conditions: (1) it passes the detection phase with 50 reruns, (2) it does not cause any other test to fail, and (3) it does not alter the essential test logic upon manual examination, confirmed by an experienced Java developer after reading the patches line by line. All experiment scripts and generated

patches are publicly available in the artifact repository for verification.

To compare the results against the baselines, we duplicated each detected NIO test to introduce test order dependencies and ran `iFixFlakies` and `ODRepair`—two non-LLM-based approaches designed to fix order-dependent tests without using LLMs. Table 4.1 presents an overview of the results. Among the four LLMs integrated with NIODebugger, NIODebugger-GPT-4 achieved the best performance, producing correct patches for 101 tests (58.72%), far surpassing the other models. NIODebugger-GPT-3.5-Turbo ranked second, also outperforming the open-source LLMs by a wide margin. When integrated with open-source LLMs, NIODebugger-Qwen2.5-Coder-32B-Instruct and NIODebugger-DeepSeek-Coder-33B-Instruct performed less effectively.

NIODebugger-GPT-3.5-Turbo produced unique fixes for five tests that were not addressed by other LLMs, while all patches generated by the two open-source LLMs were subsumed by GPT-4 patches. NIODebugger-GPT-4 significantly outperformed non-LLM baseline approaches, and NIODebugger-GPT-3.5-Turbo also demonstrated superior performance. iFixFlakies failed to fix any tests, as it relies on "state cleaner tests" in the same test suite, which were absent for the 172 NIO tests in our study. ODRepair fixed some tests but was less effective than GPT-based NIODebugger models, since it relies on correctly identifying the polluted state and using Randoop to generate potential cleaner tests. Moreover, Randoop-generated tests tend to be verbose, poorly organized, and filled with low-level method calls and generic variable names, hence the patches are likely less natural.

Upon further inspection, we observed that while both GPT models successfully followed the exploration prompts for all tests, the open-source models struggled to adhere to response-format-constrained prompts. For instance, Qwen2.5-Coder-32B-Instruct occasionally responded directly with `Find Method Code: {className.methodName}` without substituting `className.methodName` with an actual method name. It also sometimes elaborated on the potential problem instead of directly deciding the correct workflow. Similarly, DeepSeek-Coder-33B-Instruct frequently attempted to fix the test directly during the exploration phase. Notably, DeepSeek-Coder-33B-Instruct failed to generate responses in the correct format in the exploration phase for 89 tests, while Qwen2.5-Coder-32B-Instruct also failed for 63 tests. Such limitations fall outside the scope of NIODebugger but could be mitigated by future advancements in open-source LLMs.

To mitigate the non-determinism of LLMs at a higher temperature, we repeated the experiment twice more using the same setup and parameters as described in the previous section for both NIODebugger-GPT-4 and NIODebugger-GPT-3.5-Turbo. We run only the GPT variants, because they have a much better performance than open-source models, and have no GPU costs. NIODebugger-GPT-4 achieved 89 and 103 correct patches, compared to 101

in the original experiment, while NIODebugger-GPT-3.5-Turbo achieved 74 and 70 correct patches, compared to 68 in the original experiment. All six runs with these two proprietary models showed performance exhibiting substantial improvement over the baselines.

Overall, our findings suggest that NIODebugger, when integrated with GPT models, achieves state-of-the-art performance, outperforming baseline techniques. The poorer performance of open-source models aligns with prior work on LLM-based flaky test repair [38] and general program repair [36, 37], where GPT models consistently outperform open-source LLMs. We present a detailed breakdown of the performance of our best-performing variant, NIODebugger-GPT-4, in Table 4.2. The table includes the following columns: the project slug, the commit SHA used in our experiments, the count of non-commented, non-blank (NCNB) code lines in all Java source files, the total number of tests evaluated during the detection phase, the total number of non-commented test assertions across the entire test suite, the number of NIO tests identified by NIODebugger's detection phase, the number of NIO tests successfully fixed by NIODebugger-GPT-4, and the number of fixes that have already been accepted via pull requests (PRs) and merged into the project's main codebase.

Table 4.1: Performance of NIODebugger Variants & Potential Baselines

NIODebugger Variant or Baseline	Correct Patches
NIODebugger-GPT-4	101 (58.72%)
NIODebugger-GPT-3.5-Turbo	68 (39.53%)
NIODebugger-Qwen2.5-Coder-32B-Instruct	27 (15.69%)
NIODebugger-DeepSeek-Coder-33B-Instruct	20 (11.63%)
ODRepair	59 (34.30%)
iFixFlakies	0 (0%)

4.4 RQ2: CONTRIBUTIONS TO REAL-WORLD SOFTWARE

To assess the developer's attitude towards NIODebugger in real-world software development, we submitted PRs for all 101 patches generated by our best-performing variant, NIODebugger-GPT-4. We made only minor adjustments to ensure the patches passed Checkstyle, without making logical modifications. Of these, 58 patches were accepted, 1 was rejected, and the remaining 42 patches are still pending.

Of the 58 patches accepted, 52 were accepted directly, while 6 required changes before acceptance. In 5 of these 6 cases, the logic of the patches was approved, requiring only minor adjustments. These adjustments included adding a `try - finally` block or altering when state cleanup occurs within the test. The other change suggested by the developer

Table 4.2: Detailed View of Projects with NIO Tests and Fixer Performance of NIODebugger-GPT-4

Project	SHA	NCNB Lines	Total Tests	NIO	Fixed	Accepted
apache/dubbo	20f252d	287009	6415	34	19	19
apache/hadoop	ecf665c	1929672	10874	31	14	7
kiegroup/jbpm	1558f0d	295347	3675	23	11	0
sismics/docs	afa7885	24600	74	21	17	17
apache/cxf	eea3c9b	695144	9528	15	3	3
alibaba/COLA	1a8c433	11654	69	12	11	0
brianfrankcooper/YCSB	ec3eb9c	25434	76	12	12	0
apache/wicket	58d953e	220299	2811	6	5	5
spring-cloud/spring-cloud-netflix	2a8b7ed	12259	235	4	1	1
ebean-orm/ebean	d034821	222188	1069	2	2	2
stleary/JSON-java	8983ca6	14263	647	2	2	2
apache/rocketmq	b37d283	242774	1676	2	1	1
apache/tika	f78dc99	173227	1982	1	1	0
apache/tinkerpop	8bb5d16	172540	25269	1	1	1
eclipse-vertx/vert.x	0eb288b	140880	4849	1	1	0
apache/incubator-kie-optaplanner	8c2fb1e	208565	3963	1	0	N/A
Red5/red5-server	eb75c16	64046	170	1	0	N/A
spring-projects/spring-retry	9442435	11468	387	1	0	N/A
stanfordnlp/CoreNLP	2460079	619842	1459	1	0	N/A
winder/Universal-G-Code-Sender	445cd19	92947	750	1	0	N/A

involved making a non-idempotent function under test idempotent directly, thus addressing issues outside the scope of the test method.

Particularly noteworthy is that the rejected PR [55] does not stem from incorrectness of the patch generated by NIODebugger. Figure 4.1 illustrates our patch. Specifically, the patch deploys a verticle to the global set `ReferenceSavingMyVerticle.myVerticles` but does not clean up after deployment. Consequently, in the second execution, `myVerticles` still contains a verticle from the previous run with a different deployment ID than the currently deployed one. This discrepancy causes the assertion `assertEquals(deploymentId, myVerticle.deploymentID)` to fail in one of the iterations of the `forEach` loop. In this scenario, the developer acknowledges the patch's correctness but rejects our PR because they do not consider such state pollution hazardous.

4.5 RQ3: CONTRIBUTIONS OF KEY COMPONENTS

This section examines the contribution of two key components of NIODebugger through an ablation study on NIODebugger-GPT-4, the best performing variant. We isolate source code extraction used for context exploration and dynamic analysis used during the detection

Figure 4.1: DeploymentTest.java in eclipse-vertx/vert.x

```
public void testDeployClass() {
+   ReferenceSavingMyVerticle.myVerticles.clear();
    vertx.deployVerticle(//...).onComplete(onSuccess(deploymentId -> {
      ReferenceSavingMyVerticle.myVerticles.forEach(myVerticle -> {
        assertEquals(deploymentId, myVerticle.deploymentID);
        assertEquals(config, myVerticle.config);
        assertTrue(myVerticle.startCalled);
      });
    }));
    // ...
}
```

phase and perform an ablation study. Table 4.3 presents the results, where "RSCE" denotes relevant source code extraction and "DA" refers to dynamic analysis. Each row indicates the number of correct patches generated.

Table 4.3: Ablation Study of NIODebugger-GPT-4

Technique	Correct Patches
NIODebugger-GPT-4 without RSCE & DA	26 (15.12%)
NIODebugger-GPT-4 without RSCE	42 (24.42%)
NIODebugger-GPT-4 without DA	55 (31.98%)
NIODebugger-GPT-4	101 (58.72%)

It is evident that both dynamic analysis and relevant source code extraction are crucial for of NIODebugger, while relevant source code extraction is more indispensable.

We provide an example illustrating how NIODebugger cannot generate a patch without relevant source code extraction. Figure 4.2 depicts an NIO test in`apache/rocketmq`. The test fails in subsequent runs because it registers a `NothingFilter` without unregistering it. In the repeated run, an error occurs when registering the filter since a filter with the same identity already exists. Without relevant source code extraction, although the fixer LLM identifies the source of state pollution and guesses that a method `unRegister()` exists in `FilterFactory`, it fails to recognize that `unRegister()` requires its parameter as a filter-specific string, namely `"Nothing"` as defined in `NothingFilter`. As a result, the fixer mistakenly extracts `new NothingFilter()` as a local variable and passes it to `unRegister()`, resulting in a compilation error. Conversely, the relevant source code extraction rountine guides the exploration of the `FilterFactory` class, enabling a fix accepted by the developers [56].

For a more detailed overview, Table 4.4 presents the number of tests with which each agentic workflow is utilized, as well as the number of NIO tests successfully fixed when the

Figure 4.2: FilterSpiTest.java in apache/rocketmq

```
public void testRegister() {
    FilterFactory.INSTANCE.register(new NothingFilter());
    // Other logic
+   FilterFactory.INSTANCE.unRegister("Nothing");
}
```

workflow is employed by our best-performing variant, NIODebugger-GPT-4. "Find Class Code" was the most common workflow, followed by "Find Method Code". Both of the workflows lead to a significant portion of correct patches. Less frequent workflows, such as "Find Relevant File" and "Find Hypothesized Method", also contributed to correct fixes. Additionally, NIODebugger-GPT-4 decides that no additional code is needed in 34 cases, resolving 23 of them.

Table 4.4: Summary of Workflow Usage and Effectiveness

Workflow	# Tests Invoked	# Correct Patches
Find Class Code	75	48
Find Method Code	46	23
Directly Fixable	34	23
Find Relevant File	13	5
Find Hypothesized Method	4	2

We also provide an example where NIODebugger cannot generate a patch without dynamic analysis. Figure 4.3 shows an NIO test in the `hadoop` project. The method `registerSubCluster()` registers a call with a latency, while `getLatencySucceededCalls()` returns the mean latency of all registered successful calls. Starting from a fresh state, the assertion at line 6 passes in the first run with a mean latency of `100`, as only one call with latency `100` (at line 5) is recorded before the assertion. However, in the second execution of the test, the recorded history includes two previous calls (with latencies of `100` (at line 5) and `200` (at line 9)) from the first execution, and one call with latency `100` from the second execution. Consequently, the `getLatencySucceededCalls()` method in the assertion returns a mean latency of `133.33` (mean of 100, 200, and 100) in the first rerun, followed by `125` and `120` in subsequent runs. The changing error message provides valuable insights into state pollution accumulation. While NIODebugger without dynamic analysis generates an incorrect patch that assumes `getLatencySucceededCalls()` is simply incremented by 100 after the `registerSubCluster(100)` call, it successfully generates a patch for an approved PR [57] when dynamic analysis is enabled, as shown in Figure 4.3. This example underscores NIODebugger's capability to address complex NIO tests by analyzing stack traces from multiple

Figure 4.3: TestFederationStateStoreClientMetrics.java in apache/hadoop

```
public void testSuccessfulCalls() {
    long totalGoodBefore = FederationStateScoreClientMetrics.getNumSucceededCalls();
+   long meanLatencyBefore = FederationStateScoreClientMetrics.getLatencySucceededCalls
    ↪ ();
    //...
    goodStatesStore.registerSubCluster(100);
-   assertEquals(100, FederationStateScoreClientMetrics.getLatencySucceededCalls());
+   assertEquals((totalGoodBefore * meanLatencyBefore + 100) / (totalGoodBefore + 1),
    ↪ FederationStateScoreClientMetrics.getLatencySucceededCalls());
    //...
    goodStatesStore.registerSubCluster(200);
    //...
}
```

test runs.

CHAPTER 5: THREATS TO VALIDITY

We identify several potential threats to the validity of our approach and evaluation, and describe how we addressed each:

1. **Data Leakage**: Our evaluation includes closed-source GPT models with unknown training data, which may include the projects we analyze. Although we evaluate NIO tests with no prior fixes, the model's prior exposure to the codebase might lead to misleadingly high performance. However, the ablation study in RQ3 addresses this concern by showing that the success rate of GPT-4-generated patches decreases significantly when dynamic analysis or relevant source code extraction are removed.

2. **Validity of Patches**: There is a risk that patches generated by NIODebugger may not be ideal. To mitigate this, we submitted PRs for all patches. Out of 59 patches reviewed by developers, 58 were accepted. The one rejection was not due to incorrectness but rather the developer's reluctance to address state pollution.

3. **Scalability**: Our evaluation is limited to Java projects, raising concerns about the applicability to NIO tests in other programming languages. We address this by discussing the language-specific aspects of our framework and explaining why extending support to multiple languages is practical.

4. **Non-determinism**: The LLM response is inherently non-deterministic, particularly due to the higher temperatures. To mitigate this, we repeated the experiment twice for the two best-performing variants and observed consistently better performance than the baselines.

CHAPTER 6: LIMITATIONS

While NIODebugger demonstrates promising results, it has several notable limitations. These limitations can be categorized into three major areas: (1) efficiency of detection, (2) limitations in the agentic workflow, and (3) restrictions in prompt design and handling of inherently non-idempotent operations.

- **Scalability and Efficiency of Detection at Test-Level Granularity.**

  The current implementation of NIODebugger is not optimized for detection at the test-level granularity. Running each individual test in isolation introduces substantial overhead, especially in large-scale projects with hundreds or thousands of tests. This overhead stems primarily from the cost of repeatedly launching new JVM instances and setting up isolated test runners for each test. As a result, we conduct our evaluation at the test suite granularity—executing entire suites repeatedly to identify NIO behavior.

  However, this approach trades off precision for performance:

  - **False Positives:** Tests that are merely order-dependent (OD) may be mislabeled as non-idempotent when observed in aggregate. To address this, we re-execute any NIODebugger-labeled tests in isolation to verify their behavior. This post-filtering helps reduce false positives but adds extra steps to the pipeline.
  - **False Negatives:** More critically, false negatives can occur when NIO tests are masked by other tests in the same suite. For instance, if a suite includes tests `t3` and `t4`, and `t3` is self-polluting (NIO), but `t4` coincidentally resets the polluted state, the suite as a whole will appear idempotent. Such interactions obscure the presence of latent polluters and lead to missed detections.

  Future iterations of NIODebugger should explore opportunities for parallelism and more efficient isolation techniques—such as containerization or JVM reuse—to make per-test execution more scalable and practical for large projects.

- **Crudeness and Limitations of the Agentic Workflow.**

  The reflection-based patch generation in NIODebugger relies on a simple retry mechanism: the agent is given a failure message and the most recent (incorrect) patch and prompted to try again. While this mechanism enables basic self-correction, it leaves much room for sophistication:

- **Lack of Multi-Patch Exploration:** The agent currently generates only one candidate patch per iteration. Advanced techniques like information retrieval, patch ranking, or even ensemble generation could allow the agent to explore multiple alternatives simultaneously and select or refine the most promising candidates.
- **No Progress Tracking:** There is no mechanism for assessing whether the agent's patch iterations are converging toward a correct solution. Incorporating lightweight post-checkers or similarity metrics could help evaluate progress over time and avoid cycling through semantically similar but incorrect patches.
- **Distinguishing Plausible vs. Correct Patches:** The fixer terminates once a patch is deemed plausible—i.e., the code compiles and the test becomes idempotent. However, plausibility does not imply correctness. In practice, we observed several cases where generated patches unintentionally altered the test logic, or simply added static flags to skip the focal test in reruns. Future enhancements could include basic formal verification, symbolic execution, or dynamic assertion checking to ensure correctness beyond compilability.

Overall, our agentic workflow remains a baseline. Integrating more intelligent decision-making and feedback mechanisms will be critical to improving its reliability.

- **Limitations in Prompt Design and Resetting of Irreversible State.**

  The prompts used in NIODebugger are centered around the idea of identifying and resetting polluted shared state. This works well when the polluted state is resettable—such as lists, counters, temporary files, or custom objects with proper resetting methods. However, in many real-world applications, state changes are:

  - **Inherently Non-idempotent:** Operations like HTTP POST requests, loading classes into a Java classloader, or initializing unmodifiable collections are fundamentally non-reversible. Once performed, these actions cannot be "undone" cleanly without violating application semantics.
  - **Non-resettable in Practice:** Even when state could theoretically be reset, doing so may be brittle, non-trivial, or even dangerous—for example, cleaning up system-level resources, modifying global registries, or resetting external service connections.

  In such cases, the only feasible solution may be to isolate each test in a clean environment by instantiating a new object or process for every execution. This shifts

the resolution strategy from "resetting polluted state" to "avoiding shared state altogether."

However, NIODebugger is currently not designed to attempt such "non-state-resetting" patches. It lacks the ability to:

- Automatically distinguish between resettable and non-resettable state
- Adjust its patch generation strategy accordingly
- Prompt itself to create structurally different fixes (e.g., object recreation rather than field resetting).

In general, tackling this limitation requires identifying and handling irreversible state pollution, which remains an open research challenge.

CHAPTER 7: CONCLUSION

In conclusion, this thesis presents NIODebugger, the first framework that effectively addresses non-idempotent-outcome (NIO) flaky tests using an LLM-based agent. By integrating dynamic analysis during detection phase and agent-based relevant source code extraction, NIODebugger demonstrates strong performance in detecting and repairing NIO flaky tests in popular open-source projects, with numerous patches accepted by the community. While NIODebugger can interface with various off-the-shelf LLMs, it achieves more promising results with proprietary GPT-based models. The challenge of enabling state-of-the-art open-source LLMs to generalize to agentic workflows remains well recognized and continues to be an active area of research. We anticipate that future advancements in open-source LLMs will enhance their effectiveness when integrated with NIODebugger.

REFERENCES

[1] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *FSE*, 2014.

[2] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM TOSEM*, 2021.

[3] W. Zheng, G. Liu, M. Zhang, X. Chen, and W. Zhao, "Research progress of flaky tests," in *SANER*, 2021.

[4] S. Habchi, G. Haben, J. Sohn, A. Franci, M. Papadakis, M. Cordy, and Y. L. Traon, "What made this test flake? pinpointing classes responsible for test flakiness," in *ICSME*, 2022.

[5] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. Le Traon, "FlakyCat: Predicting flaky tests categories using few-shot learning," in *AST*, 2023.

[6] G. Haben, S. Habchi, J. Micco, M. Harman, M. Papadakis, M. Cordy, and Y. Le Traon, "The importance of accounting for execution failures when predicting test flakiness," in *ASE*, 2024.

[7] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in JavaScript," in *ICSME*, 2022.

[8] W. Lam, K. Muşlu, H. Sajnani, and S. Thummalapenta, "A study on the lifecycle of flaky tests," in *ICSE*, 2020.

[9] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: the developer's perspective," in *ESEC/FSE*, 2019.

[10] E. Kowalczyk, K. Nair, Z. Gao, L. Silberstein, T. Long, and A. Memon, "Modeling and ranking flaky tests at Apple," in *ICSE-SEIP*, 2020.

[11] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, "Root causing flaky tests in a large-scale industrial setting," in *ISSTA*, 2019.

[12] Y. Qin, S. Wang, K. Liu, B. Lin, H. Wu, L. Li, X. Mao, and T. Bissyandé, "Peeler: Learning to effectively predict flakiness without running tests," in *ICSME*, 2022.

[13] S. Fatima, T. A. Ghaleb, and L. Briand, "Flakify: A black-box, language model-based predictor for flaky tests," *IEEE TSE*, 2023.

[14] M. Gruber, M. Heine, N. Oster, M. Philippsen, and G. Fraser, "Practical flaky test prediction using common code evolution and test history data," in *ICST*, 2023.

[15] A. Alshammari, C. Morris, M. Hilton, and J. Bell, "FlakeFlagger: Predicting flakiness without rerunning tests," in *ICSE*, 2021.

[16] G. Pinto, B. Miranda, S. Dissanayake, M. d'Amorim, C. Treude, and A. Bertolino, "What is the vocabulary of flaky tests?" in *MSR*, 2020.

[17] A. Ahmad, O. Leifler, and K. Sandahl, "An evaluation of machine learning methods for predicting flaky tests," in *International Workshop on Quantitative Approaches to Software Quality (APSEC QuASoQ)*, 2020.

[18] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "iDFlakies: A framework for detecting and partially classifying flaky tests," in *ICST*, 2019.

[19] A. Gyori, B. Lambeth, A. Shi, O. Legunsen, and D. Marinov, "NonDex: A tool for detecting and debugging wrong assumptions on Java API specifications," in *FSE Tool Demo*, 2016.

[20] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, "DeFlaker: Automatically detecting flaky tests," in *ICSE*, 2018.

[21] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "What do developer-repaired flaky tests tell us about the effectiveness of automated flaky test detection?" in *AST*, 2022.

[22] A. Shi, W. Lam, R. Oei, T. Xie, and D. Marinov, "iFixFlakies: A framework for automatically fixing order-dependent flaky tests," in *ESEC/FSE*, 2019.

[23] R. Wang, Y. Chen, and W. Lam, "iPFlakies: A framework for detecting and fixing python order-dependent flaky tests," in *ICSE Tool Demo*, 2022.

[24] C. Li, C. Zhu, W. Wang, and A. Shi, "Repairing order-dependent flaky tests via test generation," in *ICSE*, 2022.

[25] P. Zhang, Y. Jiang, A. Wei, V. Stodden, D. Marinov, and A. Shi, "Domain-specific fixes for flaky tests with wrong assumptions on underdetermined specifications," in *ICSE*, 2021.

[26] Y. Pei, J. Sohn, S. Habchi, and M. Papadakis, "Non-flaky and nearly optimal time-based treatment of asynchronous wait web tests," *ACM TOSEM*, 2025.

[27] S. Zhang, D. Jalali, J. Wuttke, K. Muşlu, W. Lam, M. Ernst, and D. Notkin, "Empirically revisiting the test independence assumption," in *ISSTA*, 2014.

[28] JUnit Team, "JUnit," 2024. [Online]. Available: https://junit.org/

[29] A. Wei, P. Yi, Z. Li, T. Xie, D. Marinov, and W. Lam, "Preempting flaky tests via non-idempotent-outcome tests," in *ICSE*, 2022.

[30] C. S. Xia and L. Zhang, "Less training, more repairing please: revisiting automated program repair via zero-shot learning," in *ESEC/FSE*, 2022.

[31] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *ICSE*, 2023.

[32] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT," in *ISSTA*, 2024.

[33] J. A. Prenner, H. Babii, and R. Robbes, "Can OpenAI's Codex fix bugs? an evaluation on QuixBugs," in *International Workshop on Automated Program Repair (APR)*, 2022.

[34] S. D. Kolak, R. Martins, C. L. Goues, and V. J. Hellendoorn, "Patch generation with language models: Feasibility and scaling behavior," in *Deep Learning for Code Workshop (DL4C)*, 2022.

[35] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," in *ICSE*, 2025.

[36] C. Lee, C. S. Xia, J. tse Huang, Z. Zhu, L. Zhang, and M. R. Lyu, "A unified debugging approach via LLM-Based multi-agent synergy," *arXiv:2404.17153*, 2024.

[37] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *ISSTA*, 2024.

[38] Y. Chen and R. Jabbarvand, "Neurosymbolic repair of test flakiness," in *ISSTA*, 2024.

[39] S. Fatima, H. Hemmati, and L. C. Briand, "FlakyFix: Using large language models for predicting flaky test fix categories and test code repair," *IEEE TSE*, 2024.

[40] Y. Chen and R. Jabbarvand, "Can ChatGPT repair non-order-dependent flaky tests?" in *Proceedings of the 1st International Workshop on Flaky Tests (FTW)*, 2024.

[41] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, "Detecting assumptions on deterministic implementations of non-deterministic specifications," in *ICST*, 2016.

[42] W. Lam, "International Dataset of Flaky Tests (IDoFT)," 2020. [Online]. Available: https://github.com/TestingResearchIllinois/idoft

[43] R. Just, D. Jalali, and M. D. Ernst, "Defects4J: A database of existing faults to enable controlled testing studies for Java programs," in *ISSTA Tool Demo*, 2014.

[44] Snyk, "JVM ecosystem report 2021," 2021. [Online]. Available: https://snyk.io/reports/jvm-ecosystem-report-2021/

[45] JetBrains, "The State of Developer Ecosystem 2021," https://www.jetbrains.com/lp/devecosystem-2021/java/, 2021.

[46] The Apache Software Foundation (ASF), "Maven surefire plugin," https://maven.apache.org/surefire/, 2024, accessed: 2024-07-31.

[47] A. W. Appel, *Modern Compiler Implementation in C*. Cambridge University Press, 1997.

[48] JavaParser community, "JavaParser," 2024. [Online]. Available: https://javaparser.org/

[49] Python Software Foundation, "ast: Abstract Syntax Trees in Python," 2024. [Online]. Available: https://docs.python.org/3/library/ast.html

[50] Clang Team, *Clang: A C language family frontend for LLVM*, 2024. [Online]. Available: https://clang.llvm.org/

[51] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, 1966.

[52] D. Guo, Q. Zhu, D. Yang, Z. Xie, K. Dong, W. Zhang, G. Chen, X. Bi, Y. Wu, Y. K. Li, F. Luo, Y. Xiong, and W. Liang, "DeepSeek-Coder: When the large language model meets programming – the rise of code intelligence," *arXiv:2401.14196*, 2024.

[53] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, Y. Fan, Y. Zhang, A. Yang, R. Men, F. Huang, B. Zheng, Y. Miao, S. Quan, Y. Feng, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-Coder technical report," *arXiv:2409.12186*, 2024.

[54] Aider-AI, "Aider LLM Leaderboards: Code editing leaderboard," https://aider.chat/docs/leaderboards, 2024.

[55] Pull Request #5190 in the Vert.x GitHub Repository, "Fixed non-idempotent test 'DeploymentTesttestDeployClass'," https://github.com/eclipse-vertx/vert.x/pull/5190, April 2024.

[56] Pull Request #8093 in the RocketMQ GitHub Repository, "[ISSUE 8092] Fixed non-idempotent test," https://github.com/apache/rocketmq/pull/8093, May 2024.

[57] Pull Request #6793 in the Hadoop GitHub Repository, "YARN-11694. Fixed non-idempotent unit tests in the Yarn Module," https://github.com/apache/hadoop/pull/6793, May 2024.